10 Technical Interview Questions & Answers — FinDoc-AuditEngine v3.0

Prepared For: Linkific Day 29 & Day 30 Technical Review & Mentor Panel

Candidate: Shri Sanjaykumar V (`M.Tech Integrated in Software Engineering, VIT Vellore`)

Verified Test Suite Status: Day 29: `28 passed` (`python -m pytest tests -q`) | Day 30: `33 passed` (`python -m pytest tests -q`) — *all collected tests passed*

Question 1: System Architecture

Q: Walk us through the end-to-end architecture of `FinDoc-AuditEngine v3.0` when an invoice is submitted.

Answer:

When `POST /api/v1/audit/process` receives an invoice and Purchase Order payload, it executes a 4-stage sequential pipeline:

1. **Pydantic v2 Schema Validation (`app/models.py`):** Validates data types, positive quantities/prices, date ordering, and verifies that line-item subtotals plus tax match `total_amount_usd` within a `$0.05` tolerance.
2. **Pre-Inference Regex PII Masking (`app/core/privacy.py`):** `FinancialPrivacyScrubber` scans contact/tax fields against 6 configured regex patterns (PAN, Aadhaar format, Bank Account, Credit Card, Phone, Email) and replaces matches with indexed surrogates (`[PAN_01]`).
3. **Deterministic 3-Way Reconciliation (`app/services/matcher.py`):** `ReconciliationMatcher` compares each invoice SKU against the PO unit price and Goods Receipt Note (GRN) net accepted quantity (`received - rejected`).
4. **7-D Isolation Forest & Approval Routing (`app/services/anomaly_detector.py` & `app/services/router.py`):** `AnomalyScorer` computes a `0–100` engineered anomaly-risk indicator and perturbation feature attributions, after which `ApprovalRouter` assigns one of 5 governance tiers and appends the result to the prototype's in-memory `AUDIT_LEDGER` list.

Question 2: Why Isolation Forest Was Selected

Q: Why did you choose an unsupervised `IsolationForest` model alongside deterministic 3-way matching rules?

Answer:

Deterministic 3-way matching rules are ideal for enforcing known contract constraints—such as catching when a vendor bills `$450/unit` against a `$400/unit` PO—because accounting rules require exact arithmetic. However, deterministic rules cannot flag an invoice that matches its PO on price and quantity yet exhibits unusual multi-feature characteristics (for example, `INV-1005`: a `$49,500` invoice from a high-risk offshore jurisdiction `BZ` demanding a `Net-1` rush payment just below a `$50,000` sign-off threshold). We selected `IsolationForest` because in procurement auditing, labeled fraud examples are rare, and isolation trees effectively isolate unusual multivariate combinations without requiring labeled training classes.

Question 3: Feature Engineering & Scaling

Q: Which features are used by `AnomalyScorer`, and why did you apply ln(1 + $\text{Total}_{\text{USD}}$) and `StandardScaler`?

Answer:

We engineered 7 continuous features:

1. `log_total_usd`: **ln(1 + $\text{Total}_{\text{USD}}$)**
2. `price_variance_ratio`: price variance divided by PO subtotal
3. `qty_shortage_ratio`: missing/rejected dock units divided by billed units
4. `tax_rate_deviation`: $|\text{tax_rate} - 0.18|$
5. `line_item_concentration`: largest line item share of invoice subtotal
6. `jurisdiction_risk_weight`: country risk factor (`0.05` domestic to `1.00` offshore tax haven)
7. `payment_urgency_index`: urgency score for short payment windows (`Net-1` to `Net-3`)

We apply $\ln(1 + x)$ to compress heavy-tailed invoice dollar amounts (`$400` to `$50,000+`) and wrap the model in `StandardScaler` ($\mu=0, \sigma=1$) so that raw dollar magnitude does not dominate bounded ratios (`0.0–1.0`) when `IsolationForest` draws random split thresholds and when we compute perturbation distances.

Question 4: Risk-Score Interpretation & Synthetic Baseline Scope

Q: Is the model trained on actual historical Linkific invoices, and does a score of `99.0/100` mean a 99% probability of fraud?

Answer:

No to both—and this is an important distinction:

- **Synthetic Baseline:** The `IsolationForest` model is trained at startup on a reproducible seeded synthetic baseline of 1,000 transaction-like records (`random_state=42`) created to demonstrate financial anomaly scoring. Before any production deployment, the model and its thresholds would need to be trained, evaluated, and calibrated on representative, authorized real-world invoice data.
- **Engineered Anomaly-Risk Indicator:** The `0–100` score is a piecewise-linear mapping of the Isolation Forest `decision_function` margin combined with rule adjustments. It is an **engineered anomaly-risk indicator** used to prioritize human review, not a calibrated statistical probability of fraud.

Question 5: Local Explainability (Single-Feature Perturbation Attribution)

Q: How do you generate the feature contribution bars shown in the dashboard?

Answer:

For an invoice feature vector **x in R^7** with risk score $R(\mathbf{x})$, we iterate over each feature $k \in \{0,\dots,6\}$ and construct a counterfactual vector **x$^{(k <- mu_k)}$** where feature k is replaced by its synthetic normal baseline median **mu$_k$**. We re-evaluate the scorer and measure the marginal drop **Delta R$_k$ = max(0, R(x) - R(x$^{(k <- mu_k)}$))**. This shows the human reviewer which specific inputs (e.g., `Unit Rate Markup Ratio` or `Payment Window Urgency`) contributed most to the elevated anomaly indicator.

Question 6: Deterministic 3-Way Reconciliation Logic & Floating-Point Precision

Q: How does `ReconciliationMatcher` handle partial dock rejections and floating-point tax math?

Answer:

For each invoice line item, `ReconciliationMatcher` looks up the corresponding SKU in the Purchase Order:

- It calculates net accepted warehouse units as $Q_{\text{net}} = \max(0, Q_{\text{received}} - Q_{\text{rejected}})$. If `billed_quantity > Q_net`, it records a `QTY_SHORTAGE` variance equal to $(Q_{\text{billed}} - Q_{\text{net}}) \times P_{\text{po}}$.
- If `invoice.unit_price - po.unit_price > $0.01`, it records a `RATE_VARIANCE` equal to $Q_{\text{billed}} \times (P_{\text{inv}} - P_{\text{po}})$.
- To avoid IEEE 754 binary floating-point rounding false positives (such as `3 * 19.99 * 1.18`), line totals are rounded to 2 decimal places and checked against a configured `SUBTOTAL_TOLERANCE = 0.05` (`$0.05`).

Question 7: Pre-Inference PII Protection & Its Scope

Q: How does your privacy scrubber work, and what are its limitations?

Answer:

`FinancialPrivacyScrubber` uses 6 compiled regular expressions to mask standard patterns for Emails, Indian PAN cards (`[A-Z]{5}[0-9]{4}[A-Z]{1}`), 12-digit Aadhaar-formatted numbers, Bank Account numbers following account keywords, 16-digit Credit Cards, and 10-digit Indian phone numbers, replacing them with tokens like `[PAN_01]`.

Its limitation is that regex matching only detects strings conforming to those exact syntactic patterns—it will not catch unformatted account numbers, person names, or physical addresses, nor does `/api/v1/audit/parse-raw` perform optical OCR on scanned PDF images. In a production system, we would pair regex rules with a transformer-based Named Entity Recognition (NER) service such as Microsoft Presidio.

Question 8: API Error Handling & Validation

Q: How does the application handle invalid payloads or missing resources?

Answer:

We use two layers of defense:

1. **Pydantic v2 Validation (`HTTP 422`):** Catches negative prices, zero quantities, inverted PO/invoice dates, `grn_rejected_qty > grn_received_qty`, or subtotal mismatches exceeding `$0.05` before the route handler runs.
2. **Domain Exception Hierarchy (`app/exceptions.py`):** Custom exceptions (`ReconciliationMismatchError`, `PurchaseOrderNotFoundError`, `PrivacyScrubbingError`) inherit from `FinDocAuditException` and are caught by a global FastAPI exception handler in `app/main.py`, returning a consistent JSON structure with `error_code`, `message`, `details`, `timestamp`, and `X-Correlation-ID`.

Question 9: Automated Testing & Key Engineering Challenge

Q: How is the project tested, and what was the most significant bug you fixed during development?

Answer:

Running `python -m pytest tests -q` executes **28 tests in Day 29** and **33 tests in Day 30**, with **all collected tests passing**. The tests cover 3-way reconciliation math, anomaly score monotonicity, regex PII masking, custom exception handlers, and FastAPI endpoints.

The most instructive bug we fixed was **frontend-to-backend preset coupling**: early versions of `dashboard.js` reused the selected preset PO's item array when submitting an invoice, which meant manual edits in the UI table were ignored. We fixed this by implementing `collectLineItemsFromDOM()`, which reads `invPrice`, `poPrice`, `billedQty`, `grnRec`, and `grnRej` directly from the live HTML `<input>` elements in each row, and added a multi-line dynamic regression test to verify it.

Question 10: Current Prototype Limitations & Future Scalability

Q: What would be required to move this prototype into a real production environment?

Answer:

Four major upgrades would be required:

1. **Durable Database Persistence:** Replace the in-memory `AUDIT_LEDGER` Python list (which resets on server restart) with **PostgreSQL (`SQLAlchemy` + `asyncpg`)** and **Redis** idempotency keys to prevent duplicate invoice submissions.
2. **Real-World Model Validation & Drift Monitoring:** Retrain and evaluate the anomaly detector on authorized historical enterprise invoices, calibrate risk thresholds against verified outcomes, and track feature drift using **EvidentlyAI (Population Stability Index)**.

3. **Authentication & Security Controls:** Add **OAuth2 / JWT** authentication, role-based access control (RBAC) for approval queues, and rate limiting.
4. **Asynchronous Queueing & Optical OCR:** Use **Kafka or Celery** workers for high-volume batch ingestion and integrate optical layout extraction (**LayoutLMv3**) for scanned PDF invoices.